

Lab 1: Your First *Spring Boot App*

In this lab, you'll generate a Spring Boot project, write your first REST controller, run the application, and see a response in your browser. By the end, you'll have a working web application built from scratch.

- ▶ Type: Laboratory Exercise ▶ Dependency: Spring Web only
- ▶ No database in this lab

01 What We're Building

We're keeping this lab **intentionally simple**. No database, no JPA, no complex setup. Just one dependency — **Spring Web** — so you can focus on understanding how a Spring Boot application works.

By the end of this lab, you will:

- Generate a Spring Boot project using Spring Initializr
- Open and explore the project in your IDE
- Understand every generated file
- Write your first REST controller
- Run the application
- See a JSON response in your browser

What is Spring Web?

From the Spring Initializr description:

Spring Web — Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

This single dependency gives us everything we need:

- **Spring MVC** — the framework for handling HTTP requests and building REST APIs
- **Embedded Tomcat** — a web server built into our application (no separate installation)
- **Jackson** — automatically converts Java objects to JSON and back

 **Connection to previous lessons:** This is where Microservices (architecture), REST API (communication), Spring Framework (IoC/DI), and Spring Boot (auto-configuration) all come together. We're building a real running service.

02 Generate the Project

✓ DO THIS NOW

Open your browser and go to <https://start.spring.io>

Fill in the form with these settings (refer to the Spring Initializr lesson if you need a refresher on what each field means):

FIELD	VALUE
Project	Maven

FIELD	VALUE
Language	Java
Spring Boot	Latest stable (the one with no SNAPSHOT/M/RC label)
Group	<code>com.example</code>
Artifact	<code>hello-spring</code>
Package name	<code>com.example.hellospring</code> (verify — no hyphens!)
Packaging	Jar
Java	17 or 21 (match your installed JDK)
Configuration	Properties

Add ONE Dependency Only

✅ DO THIS NOW

Click **"ADD DEPENDENCIES"** (or press Ctrl+B), search for **"Spring Web"**, and click it to add.

You should see **only 1 dependency** on the right side: **Spring Web**.

⚠️ **Only Spring Web!** Do NOT add Spring Data JPA, MySQL Driver, or anything else for this lab. If you add JPA without a database, the app will fail to start. Keep it simple — one dependency.

Generate and Download

✅ DO THIS NOW

1 Click **GENERATE** (or press Ctrl+Enter) — a file called `hello-spring.zip` will download

2 Find the **ZIP file** in your Downloads folder

3 **Extract (unzip) it** — right-click → Extract All (Windows) or double-click (Mac)

4 You now have a folder called `hello-spring` — this is your project

03 Open the Project in Your IDE

IntelliJ IDEA — Method 1 (Open Folder)

1 Open IntelliJ IDEA

2 Click **File** → **Open**

3 Navigate to the `hello-spring` folder you extracted

4 **Before you click OK — verify you're in the right folder!** Look inside the folder contents. You should see `pom.xml`, `src` folder, `mvnw`, and `.gitignore`. If you don't see `pom.xml`, you're in the wrong folder — go one level deeper. A common mistake is selecting the parent Downloads folder or a nested folder instead of the project root.

5 Select the folder (the one that contains `pom.xml`) and click **OK**

6 If asked "Trust this project?" — click **Trust Project**

7 **Wait for Maven to finish downloading** — you'll see a progress bar at the bottom. This may take 1–2 minutes the first time. Do NOT start coding until it finishes.

⚠️ **"I opened the project but I see no Java files!"** — This usually means you opened the wrong folder. The correct folder is the one where `pom.xml` lives at the root level. If you extracted the ZIP and got `Downloads/hello-spring/hello-spring/` (double nested), you need the inner one. Close the project, do `File → Open again`, and look for the folder that has `pom.xml` directly inside it.

IntelliJ IDEA — Method 2 (Right-Click `pom.xml`)

1 **Open your file explorer** and navigate to the `hello-spring` folder

2 Find the `pom.xml` file inside the folder

3 **Right-click** on `pom.xml` → **Open with** → **IntelliJ IDEA**

4 IntelliJ will recognize it as a Maven project and open it correctly

5 If asked "Open as Project?" — click **Open as Project**

6 If asked "Trust this project?" — click **Trust Project**

7 **Wait for Maven to finish downloading**

Which method should I use? Both work. Method 2 (right-click pom.xml) is a quick shortcut — IntelliJ automatically detects the project type from the pom.xml. Method 1 (File → Open) is the standard way. Use whichever feels easier.

Eclipse / STS

1 Open Eclipse

2 Click **File** → **Import**

3 Select **Maven** → **Existing Maven Projects**

4 Browse to the `hello-spring` folder

5 Click **Finish**

6 Wait for Maven to finish downloading

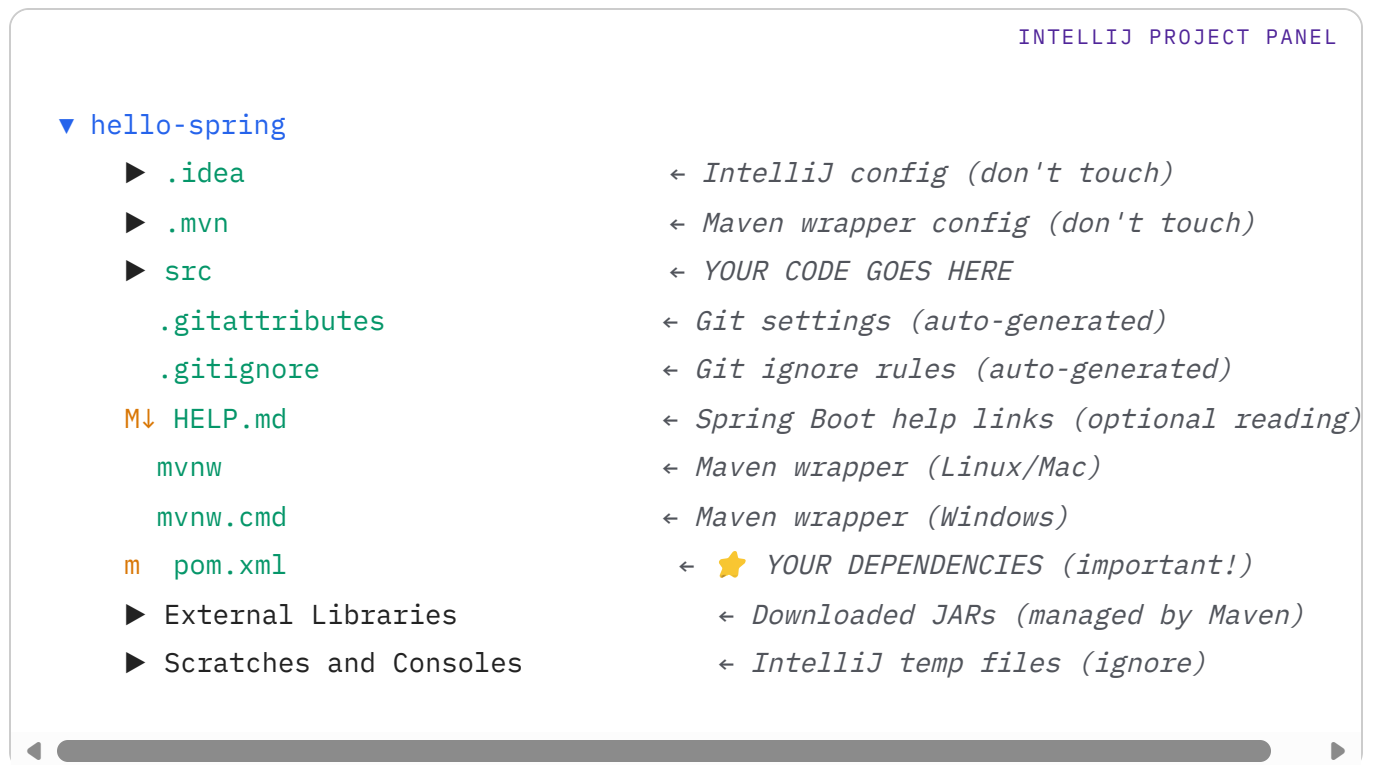
How do I know Maven is finished? In IntelliJ, the progress bar at the bottom disappears and you'll see "BUILD SUCCESS" in the Build tool window. In Eclipse, the bottom-right progress indicator stops. If you see red errors in your code after Maven finishes, try: right-click the project → Maven → Reload Project (IntelliJ) or Update Project (Eclipse).

04 Explore What Was Generated

Before we write any code, let's understand what Spring Initializr created for us.

What You Should See in IntelliJ (Project Panel)

Look at the **Project panel** on the left side of IntelliJ. Your project should look like this:



How to verify you opened the right project: You should see `pom.xml` at the root level and a `src` folder. If you don't see these, you opened the wrong folder — close and try again (see the troubleshooting tip above).

Inside the src Folder

Expand the `src` folder. This is where all your code lives:



```
▼ test
  ▼ java
    ▼ com.example.hellospring
      HelloSpringApplicationTests.java ← test file
```

The resources Folder — static, templates, and application.properties

Inside `src/main/resources/`, you'll see three things. Let's understand each one:

static/ folder

This folder is for **static files** — files that are served directly to the browser without any processing. Think of files that don't change based on who's viewing them.

What goes in `static/` :

- **CSS files** — stylesheets for web pages (`style.css`)
- **JavaScript files** — client-side scripts (`app.js`)
- **Images** — logos, icons, photos (`logo.png`)
- **HTML files** — plain HTML pages that don't need server-side processing
- **Fonts, PDFs** — any file you want users to download or view directly

For example, if you put a file called `hello.html` inside the `static/` folder, it would be accessible at `http://localhost:8080/hello.html` — Spring Boot serves it automatically, no controller needed.

 **Microservices Connection:** In microservices that serve only REST APIs (which is what we're building), the `static/` folder is usually **empty**. Static files like CSS, JavaScript, and images typically belong to the frontend application (React, Angular, Vue), not the backend API. But it's there if you ever need it.

templates/ folder

This folder is for **server-side HTML templates** — HTML pages that are processed by a template engine (like Thymeleaf) before being sent to the browser. The server fills in dynamic data before the page is sent.

What goes in `templates/` :

- **Thymeleaf templates** — HTML files with dynamic placeholders that the server fills in
- **Server-rendered pages** — pages where the HTML is built on the server, not in the browser



static/ vs templates/ — What's the difference?

static/ = a printed flyer. Everyone gets the exact same flyer. The server just hands it out.

templates/ = a personalized letter. The server fills in your name, your order details, and your address before sending it. Each person gets a different version.

For this lab and for REST APIs in general: Both folders will stay **empty**. We're building a REST API that returns JSON data, not HTML pages. Our `@RestController` returns data directly — it doesn't use templates. These folders are useful when building traditional web applications with server-rendered HTML, but that's a different use case from microservices.

application.properties

Your application's **configuration file**. All settings go here: server port, database connection, logging levels, and more. We'll explore this in detail in the next section.

The test Folder

Notice that `src/` has two main sections: `main/` (your application code) and `test/` (your test code). They mirror each other:

MAIN VS TEST — MIRROR STRUCTURE

```
src/
├─ main/java/com/example/hellospring/      ← YOUR application code goes here
│   └─ HelloSpringApplication.java
├─ test/java/com/example/hellospring/      ← YOUR test code goes here
│   └─ HelloSpringApplicationTests.java
```

test/ folder

This is where your **test classes** live. Tests verify that your code works correctly. The test folder has the same package structure as main/ so that test classes can access the classes they're testing.

Spring Initializr generated one test file for you:

JAVA – HELLOSPRINGAPPLICATIONTESTS.JAVA

```
package com.example.hellospring;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class HelloSpringApplicationTests {

    @Test
    void contextLoads() {
        // This test verifies that the Spring context starts without errors
    }
}
```

LINE	WHAT IT DOES
@SpringBootTest	Tells the test framework to start the entire Spring Boot application before running the test. It creates the Spring Container, loads all beans, and configures everything — just like running the real app.
@Test	Marks this method as a test case. JUnit will run it when you execute your tests.
void contextLoads()	The test method. It looks empty, but it's actually useful — if the Spring context fails to start (bad configuration, missing bean, broken dependency), this test will fail and tell you immediately.

Don't delete this test! It may look like it does nothing, but it catches startup errors early. When you add new classes, configurations, or dependencies later, this test will be the first

thing to tell you if something is broken. Think of it as a smoke test — "can the engine even start?"

The Root-Level Files

Let's also understand the files at the root of your project (outside `src/`):

FILE / FOLDER	WHAT IT IS	SHOULD YOU TOUCH IT?
<code>.idea/</code>	IntelliJ IDEA's internal configuration files. Stores your IDE settings, run configurations, and project metadata.	✗ Never edit. Added to <code>.gitignore</code> automatically.
<code>.mvn/</code>	Maven Wrapper configuration. Contains a small JAR and properties file that lets <code>mvnw</code> download the correct Maven version.	✗ Don't touch.
<code>mvnw</code>	Maven Wrapper script for Linux/Mac . Run with <code>./mvnw spring-boot:run</code> . Lets anyone build the project without having Maven installed.	✗ Don't edit. Just use it.
<code>mvnw.cmd</code>	Maven Wrapper script for Windows . Run with <code>mvnw.cmd spring-boot:run</code> .	✗ Don't edit. Just use it.
<code>pom.xml</code>	Maven's configuration file — your dependencies, build settings, and project identity. The most important config file.	✓ Yes — you'll add dependencies here as your project grows.
<code>HELP.md</code>	A Markdown file with helpful links to Spring Boot documentation and guides. Generated by Initializr.	Optional reading. You can delete it if you want.

FILE / FOLDER	WHAT IT IS	SHOULD YOU TOUCH IT?
<code>.gitattributes</code>	Git settings for how files should be handled (line endings, binary vs text). Auto-generated.	❌ Leave as-is.
<code>.gitignore</code>	Tells Git which files to NOT track: compiled classes (<code>/target</code>), IDE files (<code>.idea</code>), OS files (<code>.DS_Store</code>). Pre-configured by Initializr.	❌ Leave as-is. You rarely need to change it.

Summary: Out of all these files, the only ones you'll actively work with are `pom.xml` (to add dependencies) and everything inside `src/main/` (your code and configuration). The rest is auto-generated infrastructure that you leave alone.

Seeing `com` → `example` → `hellospring` as Separate Folders?

Some of you might see the package displayed differently — as **separate nested folders** instead of one flat name:

EXPANDED VIEW (YOU MIGHT SEE THIS)

```

▼ java
  ▼ com
    ▼ example
      ▼ hellospring
        HelloSpringApplication

```

FLATTENED VIEW (WHAT WE SHOWED ABOVE)

```

▼ java
  ▼ com.example.hellospring
    HelloSpringApplication

```

Both are the same thing! It's just a display setting in your IDE. The package is `com.example.hellospring` either way. IntelliJ can show packages as a flat list (compact) or as nested folders (expanded). To switch between them in IntelliJ: click the  **gear icon** at the top of the Project panel → check or uncheck "**Compact Middle Packages**" or "**Flatten Packages**". Either view is fine — use whichever you prefer.

How to Verify Your Package is Correct

No matter how your IDE displays the folders, you can always verify the package by looking at the **first line of any Java class**:

✓ CHECK THIS NOW

Open `HelloSpringApplication.java` and look at the very first line of code. It should say:

JAVA – FIRST LINE OF HELLOSPRINGAPPLICATION.JAVA

```
package com.example.hellospring;
```

This `package` statement is the **source of truth**. It tells Java exactly which package this class belongs to. If this line says `com.example.hellospring`, your package is correct — regardless of how the folder tree looks in the IDE.

 **If the package line is wrong or missing:** Your classes won't be found by Spring's component scan and the app will not work. Every Java class you create must have the correct package statement as its first line. When you create new classes inside a package in your IDE, this line is added automatically — but always double-check.

The Entry Point — `HelloSpringApplication.java`

✓ LOOK AT THIS FILE NOW

In your IDE, navigate to `src/main/java/com/example/hellospring/` and open `HelloSpringApplication.java`

JAVA – HELLOSPRINGAPPLICATION.JAVA

```
package com.example.hellospring;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HelloSpringApplication {

    public static void main(String[] args) {
        SpringApplication.run(HelloSpringApplication.class, args);
    }
}
```

Let's break down every line:

LINE	WHAT IT DOES
<code>package com.example.hellospring;</code>	Declares which package this class belongs to. Matches the folder structure.
<code>import ...</code>	Imports the Spring Boot classes we need. Your IDE usually handles this automatically.
<code>@SpringBootApplication</code>	This is the magic annotation. It tells Spring to: (1) enable auto-configuration, (2) scan this package for components/beans, (3) allow bean definitions. It's three annotations combined into one.
<code>public class HelloSpringApplication</code>	A regular Java class. The name matches the file name (Java rule).

LINE	WHAT IT DOES
<code>public static void main(String[] args)</code>	The standard Java entry point. Every Java application starts with <code>main()</code> .
<code>SpringApplication.run(...)</code>	This single line starts everything: creates the Spring Container, auto-configures Spring MVC and Tomcat, scans for beans, and starts the web server.

One line does all of this: `SpringApplication.run()` → creates the `ApplicationContext` (Spring Container from lesson 3) → detects Spring Web on the classpath → auto-configures Spring MVC → starts embedded Tomcat on port 8080 → your app is live and listening for HTTP requests.

Why Are Annotations Important in Spring Boot?

In Spring Boot, **annotations are how you communicate with the framework**. Instead of writing lengthy configuration files, you put an annotation (the `@` tags) on your class or method, and Spring Boot knows what to do with it.



Think of annotations like name tags at a conference

At a conference, you wear a name tag that says your role — "Speaker", "Organizer", "Volunteer". The staff doesn't need to ask what you do — they read your tag and know immediately. In Spring Boot, annotations are those tags. You put `@RestController` on a class and Spring knows "this handles HTTP requests." You put `@Service` and Spring knows "this contains business logic." Without the annotation, Spring ignores the class — it's just a regular Java class sitting there doing nothing.

@SpringBootApplication — Three Annotations in One

`@SpringBootApplication` is not a single annotation — it's a **shortcut that combines three separate annotations**. Writing `@SpringBootApplication` is the same as writing all three of these:

```
// You write this:
@SpringBootApplication
public class HelloSpringApplication { ... }

// Which is the same as writing all three:
@Configuration
@EnableAutoConfiguration
@ComponentScan
public class HelloSpringApplication { ... }
```

ANNOTATION	WHAT IT DOES	WHY IT MATTERS
@Configuration	Marks this class as a source of bean definitions . It tells Spring "you can find configuration and bean setup here."	Without this, Spring doesn't know this class can define beans or settings.
@EnableAutoConfiguration	Tells Spring Boot to automatically configure your application based on the dependencies in your pom.xml. Found <code>spring-boot-starter-webmvc</code> ? Auto-configure Tomcat and Spring MVC. Found a database driver? Auto-configure a DataSource.	Without this, you'd have to manually configure every single component — Tomcat, Jackson, Spring MVC, etc. This is the "magic" of Spring Boot.
@ComponentScan	Tells Spring to scan this package and all sub-packages for classes with annotations like <code>@RestController</code> , <code>@Service</code> , <code>@Repository</code> , <code>@Component</code> . It finds them and registers them as Beans.	Without this, Spring wouldn't find your controllers and services — your REST endpoints wouldn't work.

⚠️ **Why @ComponentScan matters for your package structure:** @ComponentScan scans from the package where @SpringBootApplication is located — in our case, com.example.hellospring. It scans this package and everything **below** it (sub-packages like com.example.hellospring.controller). If you accidentally create a class in com.example.other (a different package, not a sub-package), Spring will **NOT** find it. Always keep your classes inside the base package or its sub-packages.

🔗 **Spring Framework Connection:** Remember IoC and DI from the Spring Framework lesson? @ComponentScan is how Spring *discovers* which classes to manage. @Configuration lets you define beans. @EnableAutoConfiguration is what makes Spring Boot different from plain Spring — it does the configuration work for you. All three concepts from the Spring Framework lesson come together in this one annotation.

The pom.xml — The Heart of Your Project

✅ LOOK AT THIS FILE NOW

Open `pom.xml` in the root of your project. This is the file that tells Maven everything about your project: who you are, what Spring Boot version to use, what libraries to include, and how to build the app.

Let's go through the **entire file** section by section. Here's what you should see:

Part 1: XML Header

XML — POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
```

What is this? This is boilerplate XML that every pom.xml needs. It declares that this is a Maven POM file using version 4.0.0 of the Maven model. **You never need to change this.** It was generated automatically and you can ignore it.

Part 2: Parent — Spring Boot's Defaults

XML — POM.XML

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.6</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
```

TAG	WHAT IT DOES
<code><parent></code>	Your project inherits from this parent project. Like a child class extending a parent class in Java.
<code>spring-boot-starter-parent</code>	The Spring Boot parent POM. It provides: default dependency versions (so you don't type versions yourself), Java compiler settings, plugin configurations, and sensible defaults for packaging.
<code><version>4.0.6</version></code>	The Spring Boot version you selected in Spring Initializr. This controls the versions of ALL Spring libraries.
<code><relativePath/></code>	Tells Maven to download the parent from the Maven repository (not look for it locally). Don't change this.

Why is this important? Because of this parent, you don't need to write version numbers on your Spring dependencies. The parent already knows which versions are compatible. This is why `spring-boot-starter-webmvc` has no `<version>` tag — the parent manages it.

Part 3: Project Identity — Your Metadata

XML — POM.XML

```
<groupId>com.example</groupId>
<artifactId>hello-spring</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name/>
<description/>
<url/>
<licenses>
  <license/>
</licenses>
<developers>
  <developer/>
</developers>
<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
```

TAG	WHAT IT DOES
<code><groupId></code>	The Group you entered in Initializr — <code>com.example</code> . Your organization's reverse domain.
<code><artifactId></code>	The Artifact you entered — <code>hello-spring</code> . This becomes your JAR file name.
<code><version></code>	<code>0.0.1-SNAPSHOT</code> — the default starting version. SNAPSHOT means "this is still in development, not a final release."
<code><name/></code> , <code><description/></code> , <code><url/></code>	Empty placeholder tags for project name, description, and URL. You can fill these in later or leave them empty. They don't affect how the app runs.

TAG	WHAT IT DOES
<code><licenses></code> , <code><developers></code> , <code><scm></code>	Empty placeholder sections for license info, developer info, and source control info. These are used when you publish your project as a library. Ignore these for now — they don't affect anything.

Don't worry about the empty tags. The `<name/>`, `<description/>`, `<licenses>`, `<developers>`, and `<scm>` sections are generated as empty placeholders. They're for project metadata when you publish a library. For our purposes, they do nothing and you can leave them as-is.

Part 4: Properties — Java Version

XML — POM.XML

```
<properties>
  <java.version>17</java.version>
</properties>
```

TAG	WHAT IT DOES
<code><java.version></code>	Tells Maven which Java version to compile with. This is the Java version you selected in Spring Initializr (17 or 21). It must match the JDK installed on your machine.

Part 5: Dependencies — The Libraries You Need

XML — POM.XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
```

```

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

```

DEPENDENCY	WHAT IT GIVES YOU	NOTES
spring-boot-starter-webmvc	Spring MVC (REST controllers) + embedded Tomcat (web server) + Jackson (Java-to-JSON conversion)	This is the one dependency you selected. No <code><version></code> needed — the parent manages it.
spring-boot-starter-webmvc-test	JUnit 5 + Mockito + Spring Test utilities for testing web controllers	Added automatically. <code><scope>test</scope></code> means it's only available in test classes — not in your main code, and not included in the final JAR.

Notice: no version numbers on the dependencies! You only see `<groupId>` and `<artifactId>` — no `<version>` tag. That's because the `<parent>` section at the top of your `pom.xml` — the `spring-boot-starter-parent` — already defines compatible versions for all Spring libraries. This is one of the biggest benefits of using the parent POM — you never have to worry about version conflicts.

"Can I use a different version of a dependency?"

Yes, you *can* — but you **shouldn't** unless you have a very specific reason. Here's why:

If you add a `<version>` tag to a dependency, it **overrides** the version managed by the parent:

XML — OVERRIDING A VERSION (NOT RECOMMENDED)

```

<!-- This forces a specific (older) version -->
<dependency>
  <groupId>org.springframework.boot</groupId>

```

```
<artifactId>spring-boot-starter-webmvc</artifactId>
<version>3.5.14</version>    <!-- ⚠ overrides the parent version -->
</dependency>
```

⚠ **Why this is dangerous:** The parent POM ensures that all Spring libraries use **compatible versions** that are tested together. If you manually set one starter to version 3.5.14 while the rest of your project uses 4.0.6, you'll likely get version conflicts — errors like `NoSuchMethodError`, `ClassNotFoundException`, or unexpected behavior at runtime. These bugs are extremely hard to diagnose because the code looks correct but the library versions don't match.

The rule: Let the parent manage all Spring dependency versions. Don't add `<version>` tags to Spring starters. If you need a specific version of a *non-Spring* library (like a database driver or a third-party tool), that's fine — those aren't managed by the parent. But for anything that starts with `spring-boot-starter-`, trust the parent.

Part 6: Build — Maven Plugins

XML — POM.XML

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

TAG

WHAT IT DOES

`<build>`

Configures how Maven builds your project.

TAG	WHAT IT DOES
<code>spring-boot-maven-plugin</code>	This plugin does two important things: (1) packages your app as an executable JAR with the embedded Tomcat inside, and (2) enables the <code>mvn spring-boot:run</code> command. Without this plugin, you'd get a regular JAR that can't run on its own.

The Full Picture

🔗 **How it all connects:** The **parent** gives you Spring Boot's defaults and version management. The **properties** set your Java version. The **dependencies** define what libraries your app uses. The **build plugin** makes your app an executable JAR. Together, these four sections turn a plain Java project into a Spring Boot application.

application.properties — Not Empty!

✅ LOOK AT THIS FILE NOW

Open `src/main/resources/application.properties` . You'll see it already has one line:

APPLICATION.PROPERTIES (AUTO-GENERATED)

```
spring.application.name=hello-spring
```

Spring Initializr pre-filled this for you based on the Artifact name you entered. It's not empty like older versions — it already knows your app's name.

What are the defaults? Even though only one property is set, Spring Boot has many defaults working behind the scenes: port **8080**, Tomcat as the web server, Spring MVC for handling requests, and Jackson for JSON conversion. You only add more properties when you want to change something.

05 Understanding application.properties

For this lab, we won't add anything to the properties file — the auto-generated `spring.application.name=hello-spring` is all we need. Spring Boot's defaults handle the rest. But let's understand what those defaults are and what you *could* configure here later.

What is a Port?

A **port** is a number that identifies a specific application running on your computer. Think of your computer as an apartment building — the **IP address** (like `localhost`) is the building address, and the **port** is the apartment number. When you go to `http://localhost:8080`, you're saying "go to this computer (`localhost`), apartment 8080 (your Spring Boot app)."

Spring Boot uses port **8080** by default. That means your app will be accessible at `http://localhost:8080`. If you ever need to change it (for example, if port 8080 is already used by another application), you would add this to your properties file:

APPLICATION.PROPERTIES – EXAMPLE (DO NOT ADD THIS NOW)

```
# Change the port to 9090 instead of the default 8080
server.port=9090
```

But for now, we leave it at the default. No need to add anything.

Common Properties You'll Use Later

As your projects grow, you'll add more properties. Here's a preview of what this file can do:

PROPERTY	WHAT IT DOES	DEFAULT
<code>spring.application.name</code>	Names your application. Shows in logs. Used by service discovery in microservices.	—
<code>server.port</code>	The port your app listens on. Change this if 8080 is already in use.	8080
<code>spring.datasource.url</code>	Database connection URL (we'll use this in Lab 2).	—

PROPERTY	WHAT IT DOES	DEFAULT
<code>logging.level.root</code>	Controls how much detail appears in the console logs.	INFO

Lines starting with # are comments. They're ignored by Spring Boot. Use them to organize and explain your configuration — your future self will thank you.

For this lab: Leave `application.properties` as it is — just the one line `spring.application.name=hello-spring`. The defaults are perfect for what we're building. We'll add database configuration and more properties in Lab 2.

06 Run It — Before Writing Any Code

Before we write a single line of code, let's **run the application as-is**. This way you'll see what a Spring Boot app looks like when it starts up, and what happens when you visit it with no controllers yet.

Start the Application

✓ DO THIS NOW — PICK ONE METHOD

Method 1 (IDE): Open `HelloSpringApplication.java` → right-click → **Run** (or click the green ► button next to the `main` method)

Method 2 (Terminal): Open a terminal in the project folder and run:

Windows: `mvnw.cmd spring-boot:run`

Mac/Linux: `./mvnw spring-boot:run`

What You Should See in the Console

Your console will show a lot of text. Don't panic — let's understand what it's telling you:

```

      .- - - - -
     /\ /  ___ ' _ _ _ _ ( _ ) _ _ _ _ \ \ \ \ \
    ( ( ) \___ / ' _ / ' _ / ' _ \ / _ ' / \ \ \ \ \
     \ /  ___ ) / | _ / / / / / / / ( _ / / ) ) ) )
      '  | ___ / . _ | _ / | _ / | _ \ __, / / / / /
     =====|_|=====|___/=/_/_/_/

:: Spring Boot ::                (v4.0.6)

```

This is the **Spring Boot banner** — it shows every time your app starts. It confirms which version of Spring Boot you're using (v4.0.6).

After the banner, you'll see several **INFO** log lines. Here's what each one means:

```
Starting HelloSpringApplication using Java 21.0.8 with PID 55384
```

Your app is starting. It tells you which Java version it's using and the **PID** (Process ID) — a number the operating system uses to identify your running application.

```
No active profile set, falling back to 1 default profile: "default"
```

We haven't configured any profiles (dev, prod, etc.) so Spring Boot uses the "default" profile. This is fine — we learned about profiles in the Spring Boot lesson and we'll use them later.

```
Tomcat initialized with port 8080 (http)
Starting service [Tomcat]
Starting Servlet engine: [Apache Tomcat/11.0.21]
```

The **embedded Tomcat** web server is starting up on port 8080. Remember — Tomcat is built into your app because of the `spring-boot-starter-webmvc` dependency. You didn't install Tomcat separately

— Spring Boot did it for you.

CONSOLE

```
Root WebApplicationContext: initialization completed in 860 ms
```

The **Spring Container** (ApplicationContext) has been created and all beans have been initialized. This is the IoC container from our Spring Framework lesson doing its job.

CONSOLE

```
Tomcat started on port 8080 (http) with context path '/'
```

Tomcat is now **live and listening**. The context path `/` means your app is at the root — <http://localhost:8080/> .

CONSOLE

```
Started HelloSpringApplication in 1.748 seconds (process running for 2.146)
```

Your app is running! It took about 1.7 seconds from start to ready. It's now a live web server waiting for HTTP requests.

Key things to check: (1) No red error text — all lines should say INFO, not ERROR. (2) You see "Tomcat started on port 8080". (3) You see "Started HelloSpringApplication". If you see all three, your app started successfully.

If you see errors: Common issues: (1) Port 8080 already in use — another app is using that port. Close it, or add `server.port=8081` to `application.properties`. (2) BUILD FAILURE — Maven hasn't finished downloading dependencies. Wait and try again. (3) Cannot find main class — make sure you're running `HelloSpringApplication.java`, not a different file.

Now Visit It in Your Browser

✓ DO THIS NOW

Open your browser and go to:

<http://localhost:8080>

The Whitelabel Error Page

You'll see something like this:

BROWSER — WHAT YOU SEE

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Thu May 29 05:55:52 2026

There was an unexpected error (type=Not Found, status=404).

Don't panic! This is NOT a bug. This is actually **good news** — it means your Spring Boot app is running and responding to requests. Let's understand what it's telling you:

PART	WHAT IT MEANS
Whitelabel Error Page	This is Spring Boot's default error page . It's called "whitelabel" because it's a plain, unbranded page — no custom design, just raw information.
"no explicit mapping for /error"	You haven't created a custom error page, so Spring Boot shows this default one.
type=Not Found	The type of error — "Not Found" means no controller is mapped to the URL you visited.

status=404

The HTTP status code. **404** means "the resource you're looking for doesn't exist." We learned this in the REST API lesson.



Why are we getting a 404?

Think of it this way: your Spring Boot app is like a **restaurant that just opened** — the building is ready, the lights are on, the kitchen is running. But there's **no menu yet**. When a customer walks in and asks for food, the staff says "sorry, we don't have anything to serve you." That's the 404.

The **building** = your Spring Boot app (running).

The **menu** = your controllers (not created yet).

The **"sorry, nothing to serve"** = the Whitelabel Error Page.

In the next step, we'll create a controller — that's like adding items to the menu.



REST API Connection: The 404 status code is exactly what we learned in the REST lesson. The browser sent a GET / request to your app. Your app has no controller mapped to /, so it returns a 404 Not Found response. Once we add a `@GetMapping("/")` or `@GetMapping("/hello")`, that 404 will turn into a 200 OK with actual content.

Your app is running! You can stop it now by pressing the red ■ stop button in your IDE, or Ctrl+C in the terminal.

07 Lab 1 Recap



What you accomplished in this lab

1. Generated a Spring Boot project using Spring Initializr with one dependency (Spring Web)
2. Opened the project in IntelliJ and waited for Maven to finish
3. Explored every file and folder: `.idea` , `.mvn` , `src/main` , `src/test` , `static/` , `templates/` , `pom.xml` , `application.properties` , `mvnw` , `.gitignore`
4. Understood the pom.xml section by section: parent, identity, properties, dependencies, build plugin
5. Learned why `@SpringBootApplication` combines three annotations and why annotations matter
6. Ran the application and read every line of the console output
7. Saw the Whitelabel Error Page and understood what 404 means — the app is running but has no endpoints yet



Next — Lab 2: Writing Your First Controller

In the next lab, we'll write actual code inside this project:

1. Create your first **REST controller** with `@RestController` and `@GetMapping`
2. See a response in your browser (no more Whitelabel Error Page!)
3. Use `@PathVariable` and `@RequestParam` to handle dynamic URLs
4. Return **JSON** from your API by returning a Java object
5. Understand how Jackson converts Java objects to JSON automatically

Submission



SUBMIT YOUR LAB

Compress your project folder into a ZIP file and submit it using the following naming format:

Lab1_EID_School.zip

Replace **EID** with your Accenture email (without @accenture.com) and **School** with your school name.

Example: If your Accenture email is `juan.dela.cruz@accenture.com` and your school is `TechUniv` , your file name would be:

`Lab1_juan.dela.cruz_TechUniv.zip`

How to Create the ZIP File

OPERATING SYSTEM	STEPS
Windows	Right-click the <code>hello-spring</code> folder → Send to → Compressed (zipped) folder → rename the ZIP file to <code>Lab1_EID_School.zip</code>
Mac	Right-click the <code>hello-spring</code> folder → Compress → rename the ZIP file to <code>Lab1_EID_School.zip</code>

⚠ **Before zipping — make sure you zip the correct folder!** The ZIP should contain the `hello-spring` folder with `pom.xml` at the root. Do NOT zip just the `src` folder or the entire Downloads folder. When someone unzips your file, they should see the `hello-spring` project ready to open in an IDE.

What should be inside the ZIP: Your entire project folder as-is — `pom.xml`, `src/`, `mvnw`, `.gitignore`, everything. You do NOT need to include the `target/` folder (compiled files) or `.idea/` (IDE settings) — but if they're included, that's fine too. The important thing is that `pom.xml` and `src/` are there.

The learning path: Microservices → REST API → Spring Framework → Spring Ecosystem → Spring Boot → Spring Initializr → **Lab 1: Your First App (today)** → Lab 2: Writing Your First Controller (next)

Lab 1: Your First Spring Boot App · Next: Lab 2 – Writing Your First Controller · Print-friendly: Ctrl+P